

Django Vue Lab

Занятие 1. HTTP, HTML, CSS

Черепанов И.Н., Филипьев В.А., 2026

Модели архитектуры клиент-сервер

Клиент - Сервер

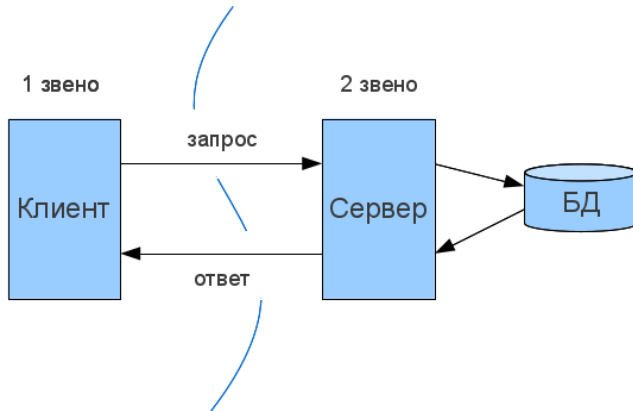


Figure 1:

Многозвенная модель

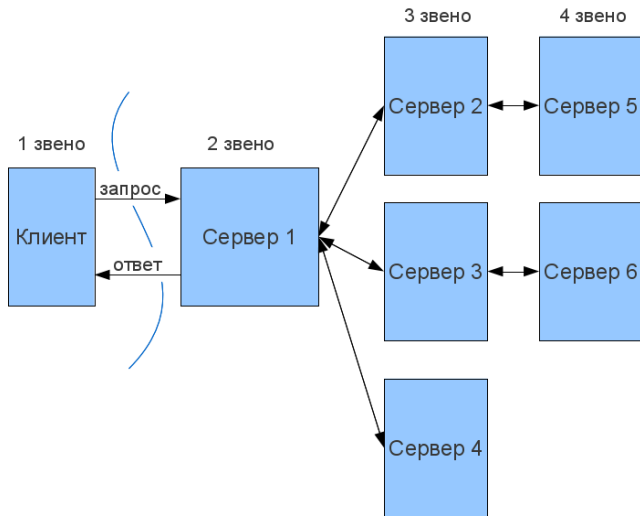


Figure 2:

Архитектуры

- «Толстый» клиент
- «Тонкий» клиент

протокол HTTP

HTTP

HTTP (англ. HyperText Transfer Protocol — «протокол передачи гипертекста») — протокол прикладного уровня передачи данных (изначально — в виде гипертекстовых документов в формате HTML, в настоящий момент используется для передачи произвольных данных).

Структура запроса

- Стартовая строка (англ. Starting line) — определяет тип сообщения;
- Заголовки (англ. Headers) — характеризуют тело сообщения, параметры передачи и прочие сведения;
- Тело сообщения (англ. Message Body) — непосредственно данные сообщения. Обязательно должно отделяться от заголовков пустой строкой.

```
GET /wiki/HTTP HTTP/1.0  
Host: ru.wikipedia.org
```


GET

GET – предназначен для получения данных с сервера. Используется для запроса содержимого указанного ресурса. С помощью метода GET можно также начать какой-либо процесс. В этом случае в тело ответного сообщения следует включить информацию о ходе выполнения процесса. Клиент может передавать параметры выполнения запроса в URI целевого ресурса после символа «?»:

```
GET /path/resource?param1=value1&m2=value2 HTTP/1.1
```

Важно: Метод GET нельзя использовать для изменения данных на сервере!!!

HEAD

HEAD – Аналогичен методу GET, за исключением того, что в ответе сервера отсутствует тело. Запрос HEAD обычно применяется для извлечения метаданных, проверки наличия ресурса (валидация URL) и чтобы узнать, не изменился ли он с момента последнего обращения. Заголовки ответа могут кэшироваться. При несовпадении метаданных ресурса с соответствующей информацией в кэше копия ресурса помечается как устаревшая

POST

POST – Применяется для передачи пользовательских данных заданному ресурсу. Например, в блогах посетители обычно могут вводить свои комментарии к записям в HTML-форму, после чего они передаются серверу методом POST и он помещает их на страницу. При этом передаваемые данные (в примере с блогами — текст комментария) включаются в тело запроса. Аналогично с помощью метода POST обычно загружаются файлы на сервер.

В отличие от метода GET, метод POST не считается идемпотентным, то есть многократное повторение одних и тех же запросов POST может возвращать разные результаты (например, после каждой отправки комментария будет появляться очередная копия этого комментария).

Сообщение ответа сервера на выполнение метода POST не кэшируется.

PUT

PUT – Применяется для загрузки содержимого запроса на указанный в запросе URI. Если по заданному URI не существует ресурс, то сервер создаёт его и возвращает статус 201 (Created). Если же был изменён ресурс, то сервер возвращает 200 (Ok) или 204 (No Content). Сервер не должен игнорировать некорректные заголовки Content-*, передаваемые клиентом вместе с сообщением. Если какой-то из этих заголовков не может быть распознан или не допустим при текущих условиях, то необходимо вернуть код ошибки 501 (Not Implemented).

Заголовок HTTP

- HOST
- Date
- Referer
- Content-Length
- Content-encoding
- User-Agent

Коды ответов сервера

- 1xx - информационные
- 2xx - успешное выполнение
- 3xx - перенаправление
- 4xx - ошибки на стороне клиента
- 5xx - ошибка на стороне сервера

200

- 200 OK («хорошо»)
- 201 Created («создано»)
- 202 Accepted («принято»)

300

- 301 Moved Permanently («перемещено навсегда»)
- 302 Moved Temporarily («перемещено временно»)

400

- 400 Bad Request («плохой, неверный запрос»)
- 401 Unauthorized («не авторизован»)
- 402 Payment Required («необходима оплата»)
- 403 Forbidden («запрещено»)
- 404 Not Found («не найдено»)
- 405 Method Not Allowed («метод не поддерживается»)
- 408 Request Timeout («истекло время ожидания»);
- 418 I'm a teapot («я — чайник»);

300

- 500 Internal Server Error («внутренняя ошибка сервера»)
- 501 Not Implemented («не реализовано»)
- 502 Bad Gateway («плохой, ошибочный шлюз»)

Стек протоколов

- HTTP,HTTPS
- TCP/UDP
- IP
- Ethernet

Простой TCP сервер

```
import socket
s = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
s.bind(('127.0.0.1', 8080))
s.listen(10)
while True:
    conn, addr = s.accept()
    request = conn.recv(1024).decode('utf8')
    url = request.rstrip("\r\n")
    file = open('/www' + str(url), 'r')
    data = file.read(1024).encode('utf8')
    conn.send(data)
    file.close()
    conn.close()
```

Язык гипертекстовой разметки HTML

HTML

HTML (от англ. HyperText Markup Language — «язык гипертекстовой разметки») — HTML (или XHTML). Язык HTML интерпретируется браузерами; полученный в результате интерпретации форматированный текст отображается на экране монитора компьютера или мобильного устройства.

Версии

- HTML 2.0 — опубликован IETF как RFC 1866 в статусе Proposed Standard (24 ноября 1995 года);
- HTML 3.0 — 28 марта 1995 года — IETF Internet Draft (до 28 сентября 1995 года);
- HTML 3.2 — 14 января 1997 года;
- HTML 4.0 — 18 декабря 1997 года;
- HTML 4.01 — 24 декабря 1999 года;
- ISO/IEC 15445:2000 (так называемый ISO HTML, основан на HTML 4.01 Strict) — 15 мая 2000 года;
- HTML5 — 28 октября 2014 года;
- HTML 5.1 начал разрабатываться 17 декабря 2012 года. Рекомендован к применению с 1 ноября 2016 года.
- HTML 5.2 был представлен 14 декабря 2017 года.

HTML

- Произвольный регистр `<BR>` `<br>`
- атрибуты без скобок: `color=red`
- непарные тэги. `<p>` вместо `<p></p>`
- Свои теги `<mysupertag>` `</mysupertag>`
- перестановки `<b><i></b></i>`

XHTML

- Только нижний регистр
- атрибуты со скобками: `color="red"`
- парные тэги. всегда `<p></p>`
- вложенность `<b><i></i></b>`
- DOCTYPE

DOCTYPE

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
```

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
```

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Frameset//EN" "http://www.w3.org/TR/html4/frameset.dtd">
```

```
<!DOCTYPE html>
```

Теги верхнего уровня

- html
- head
- body

Теги заголовка

```
<meta>  
<head>  
<link rel="stylesheet" href="./style.css">  
<script src="./jquery.js"></script>  
<meta name="description " content="Сайт о HTML">
```

Блочные теги

```
<h1> -<h6>
```

```
<p>
```

```
<hr>
```

```
<pre>
```

```
<div>
```

Строчные теги

```
<a>  
<em> <i>  
<strong> <b>  
<img src=' '>  
<span>
```

СПИСКИ

, ,

- маркированные списки

```
<ul>
  <li>one</li>
  <li>two</li>
</ul>
```

<dl>, <dt>, <dd> — списки определений

```
<dl>
<dt>HTML</dt><dd>язык разметки</dd>
<dt>CSS</dt><dd>язык описания стилей</dd>
</dl>
```

Таблицы

```
<table border="1">
  <caption>квартальный отчет</caption>
  <thead>
    <tr>
      <td>дата</td>
      <td colspan="2">доход</td>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th rowspan="2">2011-01-01</th>
      <td>100500</td>
      <td>33</td>
    </tr>
  </tbody>
</table>
```


Гиперссылки

```
<a href="URL" target="_blank" rel="nofollow">  
    
</a>
```

Якоря #:

```
<a name="chapter1">Глава 1</a>
```

Формы

```
<form method="post" action="/add/" enctype="multipart/form-  
data" target="frame3">  
  <input name="image" type="file">  
  <input name="id" type="hidden" value="3">  
  <input name="nick" type="text">  
  <input type="submit" value="Отправить">  
  <button type="submit">  
    Все равно отправить  
  </button>  
</form>
```

Элементы управления

`<input>` — текстовое поле, checkbox, radiobutton, скрытое поле, ввод пароля, выбор файла, кнопка отправки.

`<select>`, `<option>` — выпадающий селектор, множественный выбор (`multiple`)

`<textarea>` — многострочное текстовое поле.

CSS

CSS (Cascading Style Sheets — каскадные таблицы стилей) — формальный язык описания внешнего вида документа, написанного с использованием языка разметки.

```
.mid-play {  
    padding: 13px 0px 0px 13px;  
}  
  
p.inner-play a {  
    color: #3c3c3c;  
    text-decoration: underline;  
}  
  
.big-top {  
    background-image: url(/img/pc/220_130_top.gif);  
}  
  
/* комментарии */ селектор { имя_стиля1: значение1; ... }
```

Стили

- **width, height** — размеры элемента
- **margin, padding** — границы и отступы
- **display, visibility** — режим отображения
- **top, left, right, bottom** — расположение
- **background** — фон элемента
- **font** — управление шрифтом
- **text-align** — выравнивание текста

Базовые селекторы

- Универсальный селектор

```
* { margin: 0px; padding: 0px; border: 0px; }
```

- Имена тэгов

```
p { margin-top: 10px; }
```

- Имена классов (с точки)

```
.btn { border: solid 1px gray; }
```

- id тэгов (с решетки)

```
#userpic { padding: 10px }
```


Сложные селекторы

- контекстные (вложенные)

div.article a { text-decoration: underline }

- дочерние (вложенность = 1 уровень)

a > img { border: 2px }

- соседние

h2.sic + p { margin-left: 30px }

- группировка

h1, h2, h3, h4, h5 { color: red }

Псевдоклассы

- **a:visited** — посещенная ссылка
- **a:link** — непосещенная ссылка
- **div:hover** — элемент при наведении мыши
- **input:focus** — элемент при получении фокуса
- **li:first-child** — выбирает первого потомка среди множества элементов

Как определить стиль страницы

- Стили браузера
- Стили пользователя
- Стили автора
 - Во внешнем файле

```
<link rel="stylesheet" href="/style.css">
```

- В html документе

```
<style></style>
```

- Встроенные в элемент

```
<div style="display: none"></div>
```

Приоритеты стилей

- Специфичность — вычисление баллов
 - **id** – 100
 - классы и псевдоклассы – 10
 - тэги и псевдо элементы – 1
 - **ul.info ol** + **li** – 13 баллов
 - **li.red.level** – 21 балл
- Встроенный стиль: специфичность = 1000
- Расположение в коде: последний стиль
- **.inone** { display: none !important }

Типы элементов

- **display: none** — элемент невидим, не занимает места (vs visibility: hidden)
- **display: block** — элемент занимает максимальную ширину, начинается с новой строки, учитывает width, height. div, h1-h6, p – блочные.
- **display: inline** — элемент занимает минимальную ширину, и не прерывает строку, игнорирует width, height. span, img, a – строчные.
- **display: table-cell** — как ячейка таблицы

Строчные и блочные элементы

```
<div class="t">ONE</div>
<div class="t">2</div>
<span class="t">ONE</span>
<span class="t">2</span>
```

```
<style>
.t {
  width: 200px;
  height: 100px;
  background: red;
  color: white;
  margin: 5px;
  padding: 4px;
}
</style>
```

Float, clear

float — задает правила обтекания элемента

clear — отменяет обтекание начиная с элемента

Всплывание

```
<div class="outer">
<div class="sqr fl"></div> ...
<div class="clr"></div>
<div class="sqr fr"></div> ...
</div>

<style>
  .outer { float: left; width: 390px }
  .sqr { width: 100px; height: 100px }
  .fl { float: left; }
  .fr { float: right; }
  .clr { clear: both; }
</style>
```


Позиционирование

- **position: static** — обычное расположение
- **position: relative** — относительно начального местоположения на странице (смещение)
- **position: absolute** — если родитель relative, absolute или fixed – относительно родителя, иначе - относительно начала документа (страницы)
- **position: fixed** — относительно окна браузера
- **top/right/bottom/left** — отступы, могут быть отрицательными

Box model

- **position: static** — обычное расположение
- **position: relative** — относительно начального местоположения на странице (смещение)
- **position: absolute** — если родитель relative, absolute или fixed – относительно родителя, иначе - относительно начала документа (страницы)
- **position: fixed** — относительно окна браузера
- **top/right/bottom/left** — отступы, могут быть отрицательными

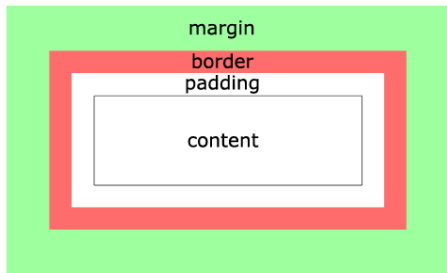


Figure 3:

Адаптивная верстка

- **media queries** — разные стили для разных экранов
- **%, vw, vh, rem** — гибкие размеры
- **flexbox и grid** — адаптивное расположение блоков
- **max-width** — ограничение ширины контента
- **адаптивные изображения**: `width: 100%; height: auto;`

CSS Framework

- Bootstrap.
- tailwindcss.
- tailwindcss.